

EdgePay Frontend Architecture & Multi-Worker Plan

A code-verified blueprint for the Admin, Merchant, and Checkout frontends on Cloudflare Workers: trust-plane isolation, Access-enforced admin APIs, SMS-corroborated checkout states, and an Astro + GSAP design system with hard performance budgets.

EdgePay Platform Engineering

Prepared following the Report 8 remediation baseline (commit 14c6690)

Table of Contents

1 · Executive Summary	3
2 · Verified API Surface and Auth Model	4
2.1 Admin surface — /api/admin/v1/*	5
2.2 Merchant surface — /api/v1/*	6
2.3 Public checkout, mobile, and gateway surfaces	6
3 · Multi-Worker Architecture	7
3.1 Why three, and what the alternative costs	8
3.2 Worker responsibility matrix	9
3.3 Repository shape and per-worker config	9
4 · Frontend-to-API Communication	11
4.1 The Service Binding trap, verified in middleware	11
4.2 Merchant BFF session pattern	12
4.3 Checkout chain and the JSON intent gap	12
5 · Screen Inventory: Admin, Merchant, Checkout	13
5.1 Admin console screens	13
5.2 Merchant panel screens	14
5.3 Checkout flow and polling discipline	14
6 · The Merchant-Write Gap and Remediation Paths	16
6.1 The gaps, with evidence	16
6.2 Path A — the target-state merchant-scoped endpoints	16
6.3 Path B — admin-mediated v1 (ships today)	17
7 · After-Payment Flow: Callbacks, SMS Corroboration, Webhooks	18
7.1 The webhook contract the UI must teach	20
7.2 Awaiting-SMS is a design problem, not a status	20
8 · Frontend Stack: Astro Islands, GSAP, and the Design System	20

8.1 Motion language with GSAP	21
8.2 Haptic response	21
8.3 Design tokens	21
8.4 Typography and locale posture	22
9 · Performance and Security Budgets	22
9.1 Security posture per worker	23
10 · Implementation Roadmap	24

1 • Executive Summary

This blueprint defines how the EdgePay platform should grow a frontend tier on top of the existing **edgepay-cf** API core: a customer-facing hosted checkout, a merchant operations panel, and a platform-admin console, each deployed as its own Cloudflare Worker. Every claim in this document was verified against two sources rather than assumed from documentation alone: the live OpenAPI document served at `/api/openapi.json` (version 0.3.0, 38 published paths) and a line-by-line reading of the repository source shipped as the Report 8 clean release, including the auth middleware chain, the checkout controller, the merchant provisioning flow, and the webhook dispatcher. Where the two sources disagree, this report says so explicitly, because several routes that earlier reviews believed to be missing are in fact implemented but simply absent from the published spec.

38 paths in the live OpenAPI 0.3.0 spec	8+ implemented routes missing from that spec	3 frontend Workers recommended	260 core tests green in workerd
--	---	---	---

The central architectural recommendation is a **three-worker split along trust planes**: **edgepay-checkout** (public and untrusted, token-scoped, holds no credentials of any kind), **edgepay-merchant** (authenticated, tenant-scoped business surface), and **edgepay-admin** (authenticated, platform-scoped, protected by its own Cloudflare Access application on a dedicated hostname). The split that matters for security is public-versus-authenticated and tenant-versus-platform, not a mechanical one-worker-per-persona; the two-worker compromise that merges admin and merchant onto one hostname is defensible but relies on path-based Access rules, which are historically the classic way an admin route gets accidentally exposed. Workers are effectively free to run on Cloudflare, so isolation here is a security decision with negligible cost, and the API core's own fail-closed Access middleware already enforces the correct call pattern from any frontend.

Code-level verification confirmed three real capability gaps that shape the UI work: merchants cannot write their own gateway credentials (the bKash personal number), merchants cannot generate a new device-pairing OTP when the 30-day provisioned one expires, and no analytics endpoint exists beyond the trial balance. Two honest remediation paths are specified: add merchant-scoped write endpoints that mirror the AES-256-GCM encrypted-at-rest pattern already used for admin-held claim credentials (target state), or ship v1 admin-mediated, where the merchant submits a request in their panel and an operator finalizes it in the admin console. This report recommends the second for launch and the first as the next backend milestone, so no screen is ever designed against an endpoint that does not exist.

Alongside this document, a working frontend kit is delivered: three production-grade interface samples built in the Astro-islands idiom with GSAP motion, smooth scroll, and haptic response (checkout, merchant dashboard, admin console), plus four architecture diagrams rendered at presentation resolution. Together they are intended to be dropped into a monorepo and wired to the real endpoints chapter by

chapter, rather than treated as throwaway mockups.

#	Decision	Rationale (verified)
D1	Three Workers, split by trust plane	checkout is public + highest traffic; separation removes any proximity between a checkout bug and merchant/admin secrets.
D2	Admin UI on a dedicated hostname with its own Access application	hostname-scoped policies avoid path-matching mistakes; the core's <code>accessAuthMiddleware</code> verifies JWTs fail-closed.
D3	Admin API calls use <code>fetch()</code> with an Access service token, never Service Bindings	bindings bypass the edge, so Access never issues the assertion the core requires; a bound call fails closed with 401.
D4	Merchant panel is a BFF holding the merchant key server-side	the browser never holds a raw API key; tenancy is still enforced by the core's <code>merchant_id</code> scoping.
D5	v1 merchant gateway/pairing writes are admin-mediated	zero backend changes needed at launch; merchant-scoped write endpoints follow as Phase 0/4 backend work.
D6	Astro static output + islands, GSAP deferred	zero-JS shell keeps checkout on budget; only interactive islands hydrate.
D7	Checkout polls status with capped backoff; webhook stays authoritative	polling is a UX nicety for the spinner; merchant systems integrate against the signed webhook.
D8	Per-surface CSP, strictest on checkout	matches the core's existing EDGE-P0-006 hardening and contains any callback-payload XSS.

Table 1.1 — The eight load-bearing decisions of this blueprint.

2 • Verified API Surface and Auth Model

The published OpenAPI document is the contract most integrators will trust, so it is worth starting with an audit of what it claims versus what the code actually mounts. The live spec describes 38 paths across six surfaces: the merchant API, the admin API, the mobile companion API, the public checkout flow, gateway webhooks, and the install wizard. Reading the route registrations in the source shows a slightly larger reality: at least eight additional implemented routes are absent from the spec, three of them administratively important (merchant list, merchant provisioning with encrypted claim tokens, and the one-time claim redemption endpoint) and three of them merchant-facing (webhook endpoint registration, listing, and deletion). Anyone designing screens from the spec alone would wrongly conclude that merchant onboarding and webhook registration must be built from scratch.

Correction worth shipping in Phase 0

The spec served at */api/openapi.json* and the Scalar reference at */api/reference* lag the implementation. Before any frontend sprint starts, regenerate the document from the router so the Merchant and Admin UIs are coded against the real contract, and so contract tests have something truthful to assert against. This is a minutes-long fix with a lasting payoff.

2.1 Admin surface — */api/admin/v1/**

Every admin route sits behind two independent gates that both have to pass. First, **accessAuthMiddleware** is mounted at */api/admin/** and verifies the Cloudflare Access JWT assertion (the *Cf-Access-Jwt-Assertion* header) against the team's JWKS endpoint: signature, issuer, audience, and expiry. The v0.2.1 policy is deliberately fail-closed — a missing or invalid assertion returns 401, an unconfigured team domain or AUD tag returns 503, and an unreachable JWKS returns 503 rather than falling back to header trust. The only non-JWT path is a dedicated break-glass service-token pair that emits a page-level observability alarm on every use. Second, **requireBearerApiAuth(['admin'])** validates an API key with admin scope, and merchant-level operations additionally require **requirePlatformAdmin**. The practical consequence for the Admin UI is that it must present both proofs on every call, which Chapter 4 turns into a concrete request recipe.

Route	Method	Scope	Purpose
<i>/merchants</i>	GET	admin + platform	List provisioned merchants (in code, not in live spec)
<i>/merchants</i>	POST	admin + platform	Provision merchant: seed gateways, pairing OTP, encrypted claim token
<i>/merchants/claim</i>	POST	admin + platform	One-time claim redemption; AES-256-GCM KV envelope (V3-004)
<i>/devices</i>	GET	admin	List paired companion devices per merchant
<i>/devices/{id}</i>	DELETE	admin	Revoke a paired device
<i>/sms-templates</i>	GET	admin	List SMS parse templates
<i>/sms-templates/{id}</i>	PUT	admin	Update regex pattern or status
<i>/sms-queues</i>	GET	admin	Recent forwarded SMS with parse status
<i>/sms-queues/{id}/retries</i>	POST	admin	Re-enqueue an SMS for parsing
<i>/domains/verifications</i>	POST	admin	Re-verify custom-domain DNS TXT record
<i>/refunds</i>	POST	admin	Create a workflow-driven (async) refund
<i>/reconcile</i>	POST	admin	Trigger the reconciliation battery manually
<i>/ledger/trial-balance</i>	GET	admin	Trial balance + DO/D1 consistency report

Table 2.1 — Admin API inventory, verified against src/controllers/admin-api.ts.

2.2 Merchant surface — /api/v1/*

Merchant routes authenticate with Bearer API keys carrying read, write, or admin scope, and every query is additionally filtered by the `merchant_id` bound to the key, which is what makes the BFF pattern in Chapter 4 safe: even a fully compromised merchant UI worker cannot read another tenant's rows, because the ceiling is enforced inside the core. Payment creation accepts an amount, currency, customer details, free-form metadata that is echoed through to webhooks, an optional pre-selected gateway, and an expiry window between sixty seconds and twenty-four hours. Refunds on this surface take the synchronous path, in contrast to the workflow-driven admin refund. The webhook registration trio (list, create, delete) is real and merchant-writable with write scope; the test-delivery endpoint even provisions a first endpoint and secret when none exists, which makes the webhook configuration screen of Chapter 5 genuinely buildable today.

Route	Method	Scope	Purpose
/payments	POST	write	Create a payment intent (amount, currency, customer, metadata)
/payments/{id}	GET	read	Fetch one payment intent
/transactions	GET	read	List transactions
/transactions/{trx_id}	GET	read	Fetch a transaction by reference
/refunds	POST	write	Refund a completed transaction (sync path)
/customers	GET	read	List customers
/api-keys	GET / POST	read / admin	List or create API keys
/gateways	GET	read	Gateway catalog: slugs, field labels, capabilities; never values
/webhooks	GET / POST / DELETE	read / write	Endpoint registration (in code, not in live spec)
/webhooks/tests	POST	write	Send a signed webhook.test event
/webhooks/deliveries	GET	read	Recent delivery log
/health	GET	public	Liveness probe

Table 2.2 — Merchant API inventory, verified against src/controllers/api.ts.

2.3 Public checkout, mobile, and gateway surfaces

The checkout surface is deliberately minimal and token-scoped: an unguessable token identifies one payment intent, and the browser that holds the link can render the page, initiate a gateway charge, submit a

TrxID for manual verification, and poll status. Per-IP rate limits guard the verification and pairing paths. The mobile companion surface authenticates with device JWTs issued after a six-digit OTP pairing, supports heartbeats, token refresh, notifications, and — critically for personal MFS payments — SMS forwarding into the parse queue. Asynchronous gateway events land on a single webhook route, deduplicated by (merchant, gateway, event_id), so a gateway retrying an event is harmless. The install wizard and bootstrap-key endpoints round out the surface and are rate-limited per IP.

Route	Method	Auth	Purpose
/checkout/{token}	GET	token	Render hosted checkout page (HTML, CSP-hardened)
/checkout/{token}/initiate	POST	token	Execute payment at the chosen gateway
/checkout/{token}/verify	POST	token	Submit customer TrxID + sender phone (alias: /submit-trx)
/checkout/{token}/callback	GET	token	Synchronous gateway redirect callback
/checkout/{token}/status	GET	token	Lightweight status poll
/api/mobile/v1/devices (or /pair)	POST	OTP	Pair device, issue access + refresh JWTs
/api/mobile/v1/sms	POST	device JWT	Forward carrier SMS to the parse queue
/webhook/{gateway}	POST	gateway auth	Async gateway events, idempotent
/install, /install/bootstrap-key	GET / POST	public	First-run wizard, per-IP limited

Table 2.3 — Checkout, mobile companion, and gateway event surfaces.

3 • Multi-Worker Architecture

The frontend tier is three new Workers deployed in front of the existing API core, each owning one trust plane, one hostname, and one blast radius. The topology keeps the money path untouched: **edgepay-cf** continues to own D1, KV, R2, the queues, the LedgerDO, and the workflows exactly as shipped, and no UI worker ever binds to a datastore directly. That preserves the audit posture the platform has earned across nine review rounds — the 260-test gate, the AES-256-GCM credential envelope, the PBKDF2-600k hashing standard — while giving each new surface an independent deploy cadence, an independent secret set, and an independent failure mode. A checkout worker incident can degrade payment-page polish without ever putting merchant sessions or the admin Access service token in the same memory.

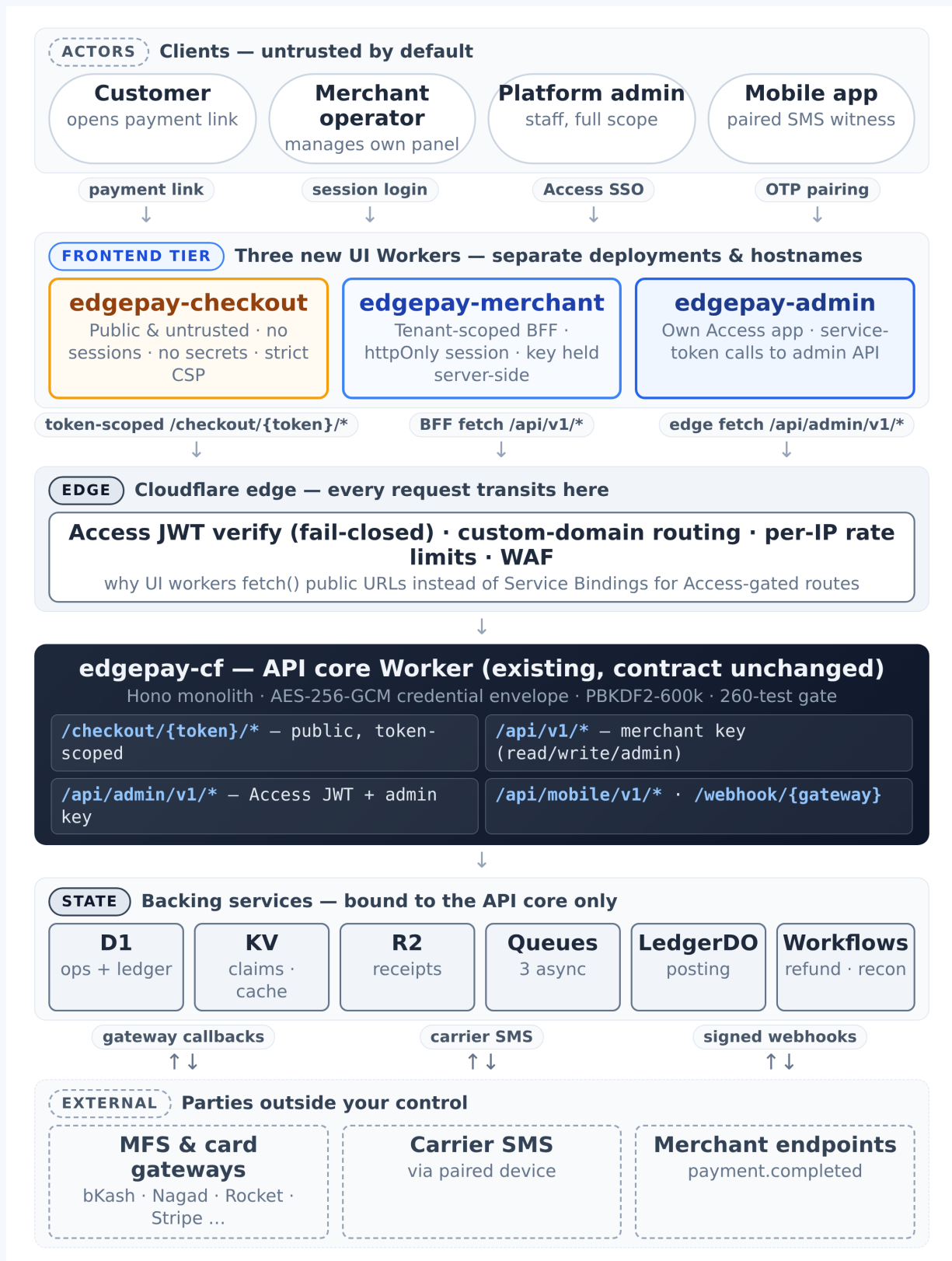


Figure 1 — Deployment topology: three frontend Workers on distinct trust planes in front of the edgepay-cf core.

3.1 Why three, and what the alternative costs

Checkout is the plane anyone on the internet touches with only a token; it receives redirect payloads from external gateways whose shape you do not fully control, and it carries the highest traffic. It must never share a deployment with anything holding merchant session state, API keys, or the Access service token — the classic failure being an XSS planted through a malformed gateway callback gaining proximity to admin credentials. Admin and merchant are both authenticated business surfaces and could technically share a Worker, with path-scoped Access rules keeping `/admin/*` locked; the two-worker variant is a legitimate compromise when minimizing operational surface early. The reason this blueprint still lands on three is that path-based Access matching is exactly the class of configuration that fails quietly: one policy that does not quite match the intended path and an admin route is public. A dedicated admin hostname carries its own Access application, needs no path matching, and the marginal cost is one more deploy target on a platform where Workers are effectively free to run — the 2-vs-3 decision is about Access-config surface, not money.

3.2 Worker responsibility matrix

Worker	Hostname (example)	Who authenticates	Secrets held	Blast radius if compromised
edgepay-checkout	pay.	nobody — token is the capability	none	a defaced payment page; no data exposure
edgepay-merchant	app.edgepay.example	merchant operator via session	per-tenant API key (KV, encrypted)	one tenant's data at most; tenancy re-checked in core
edgepay-admin	admin.edgepay.example	platform staff via Access SSO	Access service token + admin API key	platform-wide — hence dedicated hostname + alarms
edgepay-cf (core)	api.edgepay.example	all of the above + devices + gateways	JWT/APP/ENCRYPTION keys, gateway secrets	mitigated by 9 rounds of audit hardening

Table 3.1 — What each deployment is allowed to know and what it costs to lose it.

3.3 Repository shape and per-worker config

A single monorepo keeps the shared design system honest while allowing independent deploys. Astro builds each app to a static asset tree with a thin edge entry (a functions directory or a small Hono app) for the logic that must run server-side — the merchant session exchange and the admin service-token proxy. Wrangler binds each worker to its own routes and assets; the only shared resource is the API hostname they call, which is configuration, never a secret.

Monorepo layout

```

edgepay-frontend/ # monorepo (new)
apps/
checkout/ # Astro static + islands
src/pages/index.astro # payment page shell
functions/api/[...path].ts # thin token-scoped proxy (optional)
wrangler.jsonc # assets binding, routes: pay.*
merchant/
src/pages/... # dashboard islands
functions/session.ts # BFF login/session exchange
functions/api/[...path].ts # attaches Bearer key server-side
wrangler.jsonc # KV binding for sessions only
admin/
src/pages/...
functions/admin-api/[...path].ts # Access service-token proxy
wrangler.jsonc # no datastore bindings
packages/
ui/ # tokens, components, motion kit
gateway-brand/ # bKash/Nagad/Rocket brand data
package.json # workspaces, shared vitest config

```

Checkout worker configuration

```

// apps/checkout/wrangler.jsonc (excerpt)
{
  "name": "edgepay-checkout",
  "compatibility_date": "2026-08-28",
  "assets": { "directory": "./dist", "binding": "ASSETS" },
  "routes": [
    { "pattern": "pay.brand.tld", "custom_domain": true }
  ],
  "vars": { "API_ORIGIN": "https://api.edgepay.example" }
  // no kv, no dl, no r2, no queues – by design
}

```

One nuance deserves precision because it changes the Phase 1 build order: today the API core itself renders the checkout HTML at GET /checkout/{token}, complete with merchant brand color sanitization and strict CSP, and that page already works. The target state moves the page into the checkout worker for richer motion and per-brand theming, which needs the intent's public metadata (amount, currency, description, gateway list, brand) as JSON. The cheapest correct path is a small core addition — content negotiation on the existing route, returning the same data the HTML template consumes when the caller sends Accept: application/json — plus keeping the HTML renderer as a graceful fallback. Until that lands, the checkout worker can reverse-proxy the core-rendered page and layer its island scripts on top, so nothing blocks frontend work on a backend timetable.

4 • Frontend-to-API Communication

Each surface reaches the core through the public edge with a different proof of identity, and the differences are load-bearing. The checkout page authenticates nothing about the person — the token in the URL is the entire capability, scoped to one intent, rate-limited per IP. The merchant panel exchanges a write-scope API key for a server-side session exactly once, then attaches the key on every proxied call from inside the worker so the browser never sees it again. The admin console sends two proofs on every request: the Access service-token headers that the edge exchanges into a JWT assertion, and the Bearer admin-scope API key, both of which the core verifies independently and fail-closed.

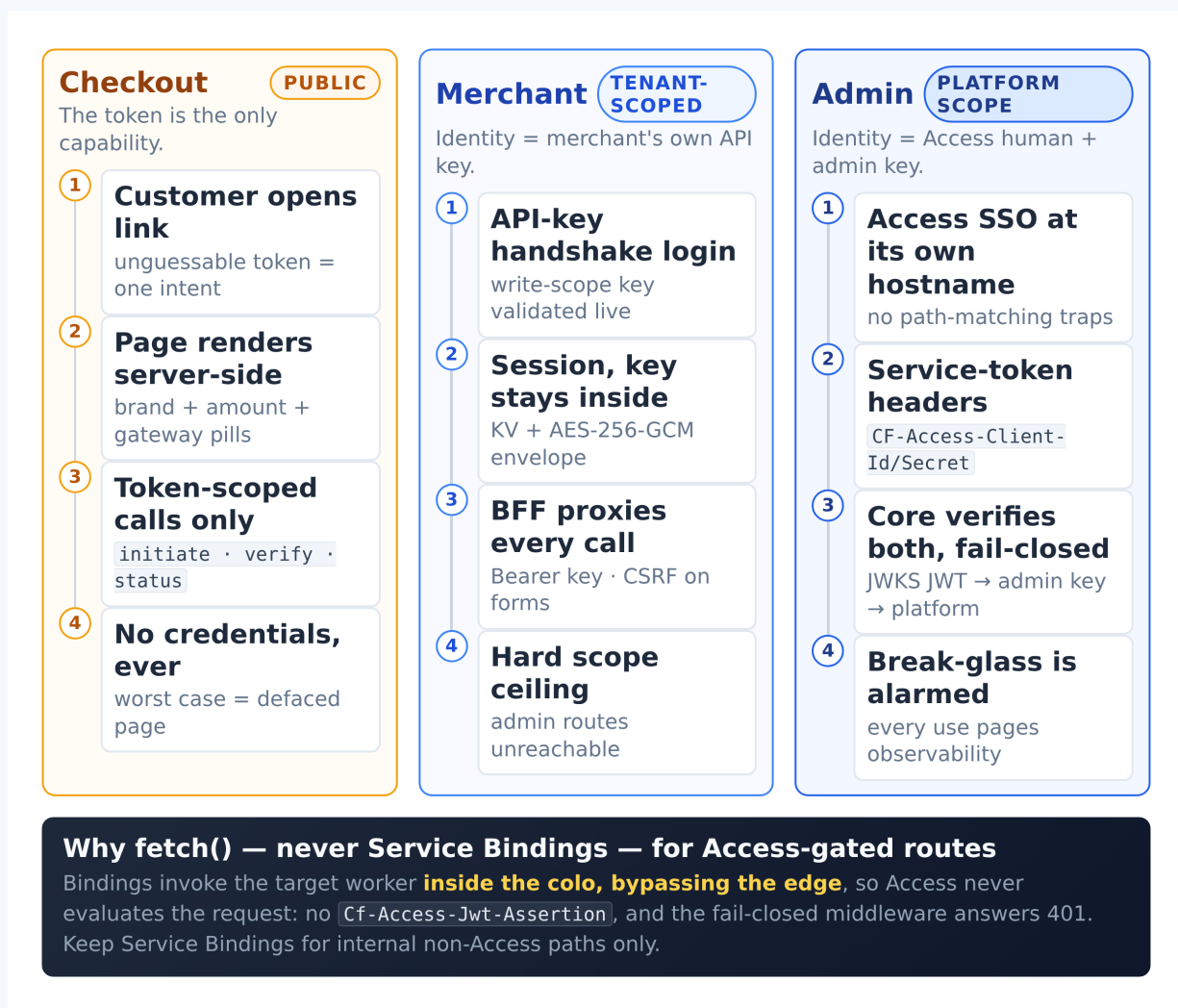


Figure 2 — The three request chains and what each one proves to the core.

4.1 The Service Binding trap, verified in middleware

It is tempting to wire the admin UI to the core with a Service Binding for latency and simplicity, and it is exactly wrong here. A bound invocation calls the target Worker directly inside the colo, bypassing

Cloudflare's edge proxy entirely — and Access policies are evaluated at the edge. The request therefore arrives with no *Cf-Access-Jwt-Assertion*, and the core's `accessAuthMiddleware` answers 401 fail-closed; the only ways around that are weakening the middleware or shipping an exception, both of which undo the hardening that v0.2.1 deliberately locked in. The architecture has quietly already made the correct decision for you: real `fetch()` to the Access-protected hostname, with the service-token headers, so the edge can do its job. Service Bindings remain the right tool where no Access policy sits in the path — most notably the future connectors Worker discussed in the Report 6 architecture line, where adapter-to-core calls are internal by construction.

Admin worker calling the Access-protected admin API

```
// apps/admin/functions/admin-api/[...path].ts (excerpt)
export const onRequest: PagesFunction = async (c) => {
  const target = new URL(c.env.API_ORIGIN + '/api/admin/v1/' + path);
  const upstream = await fetch(target, {
    method: c.request.method,
    headers: {
      'CF-Access-Client-Id': c.env.ACCESS_CLIENT_ID, // secret
      'CF-Access-Client-Secret': c.env.ACCESS_CLIENT_SECRET, // secret
      'Authorization': 'Bearer ' + c.env.ADMIN_API_KEY,
      'Content-Type': 'application/json',
    },
    body: c.request.method === 'GET' ? undefined : c.request.body,
  });
  return new Response(upstream.body, upstream);
};
```

4.2 Merchant BFF session pattern

The merchant panel never stores a raw API key in the browser. Login is a handshake: the operator submits a write-scope key once, the worker validates it with a live probe call (listing transactions with a page size of one is enough), and only then issues a session — an `httpOnly`, `Secure`, `SameSite=Strict` cookie pointing at a KV record that holds the key in an AES-256-GCM envelope, reusing exactly the encryption-at-rest pattern the core introduced for claim staging in V3-004. Every subsequent screen renders server-side: the worker reads the session, decrypts the key, attaches the Bearer header, and proxies to `/api/v1/*`. Mutating forms carry a CSRF token bound to the session. Sessions expire with the core's `SESSION_TTL_SECONDS` and are revoked on key deletion, so a leaked cookie dies on its own schedule and a revoked key kills every session that wraps it.

4.3 Checkout chain and the JSON intent gap

The checkout worker speaks only the public token-scoped routes — `initiate`, `verify`, `status` — plus the JSON intent summary once content negotiation lands (Chapter 3.3). It sets no cookies, holds no keys, and runs the strictest CSP of the three surfaces, mirroring the core's existing `EDGE-P0-006` header set. The one behavioral requirement beyond fetching is respect for rate limits: verification submissions are per-IP limited, so the UI must surface 429 responses as calm, honest messaging with a retry countdown instead of letting the user hammer a button that will keep failing.

Caller	Transport	Proof presented	Routes reachable
edgepay-checkout	fetch() via public edge	checkout token in path	/checkout/{token}/* only
edgepay-merchant	fetch() via public edge	Bearer merchant key (server-side)	/api/v1/* within key scope
edgepay-admin	fetch() via public edge	Access service token + Bearer admin key	/api/admin/v1/*
(future) connectors	Service Binding / RPC	internal trust, no Access in path	core internals only

Table 4.1 — Communication matrix: what rides the edge and what does not.

5 • Screen Inventory: Admin, Merchant, Checkout

Every screen below is mapped to endpoints that exist today, with the honest exception flags carried forward from Chapter 6. The inventory corrects the earlier external review on two points: merchant list/create and the encrypted claim flow are implemented under the admin surface, and webhook endpoint registration is implemented on the merchant surface — both merely missing from the stale published spec. Analytics remains the one genuinely thin area: the trial-balance endpoint gives balances and consistency, while revenue-over-time and success-rate charts must be aggregated client-side from the transactions list until a proper analytics endpoint ships, which is acceptable for v1 volumes and should be revisited before they grow.

5.1 Admin console screens

Screen	Endpoints	Status	Notes
Merchants list / detail	GET /api/admin/v1/merchants	Buildable	verified in code; absent from live spec — regenerate spec first
Provision merchant	POST /api/admin/v1/merchants	Buildable	returns encrypted claim token + URL, 15-minute TTL
Claim redemption log	POST /api/admin/v1/merchants/claim	Buildable	platform-admin only; one-time KV claim (V3-004)
Analytics overview	GET /api/admin/v1/ledger/trial-balance	Partial	balances + DO/D1 consistency; charts aggregate /transactions client-side
Paired devices	GET/DELETE /api/admin/v1/devices	Buildable	per-merchant list, revoke
Issue pairing OTP	POST /api/admin/v1/merchants (provision)	Indirect	new OTP only via provisioning today — see Gap G2

Screen	Endpoints	Status	Notes
SMS templates	GET/PUT /api/admin/v1/sms-templates	Buildable	regex editor + sample preview
SMS inbox / triage	GET /api/admin/v1/sms-queues, POST .../retries	Buildable	parse status + re-enqueue
Custom domains	POST /api/admin/v1/domains/verifications	Buildable	DNS TXT re-verify, KV cache invalidation
Refunds (workflow)	POST /api/admin/v1/refunds	Buildable	async — status surfaces via transactions
Manual reconciliation	POST /api/admin/v1/reconcile	Buildable	battery trigger + result view

Table 5.1 — Admin console: endpoint-by-endpoint buildability.

5.2 Merchant panel screens

Screen	Endpoints	Status	Notes
Transactions dashboard	GET /api/v1/transactions, /transactions/{trx_id}	Buildable	filters, status chips, detail drawer
Create payment link	POST /api/v1/payments	Buildable	returns hosted checkout URL + token
Customers	GET /api/v1/customers	Buildable	read-only directory
Refund a payment	POST /api/v1/refunds	Buildable	sync path with confirm step
API keys	GET/POST /api/v1/api-keys	Buildable	create needs admin-scope key (by design)
Webhook endpoints	GET/POST/DELETE /api/v1/webhooks	Buildable	verified in code — not in live spec
Webhook delivery log + test	GET /api/v1/webhooks/deliveries, POST /webhooks/tests	Buildable	signed test event validates receiver code
Gateway catalog	GET /api/v1/gateways	Buildable	field labels only — never credential values
Set bKash number / API secret	n/a	Gap G1	merchant-scoped write does not exist (Chapter 6)
Pair own phone (new OTP)	n/a	Gap G2	admin-mediated in v1 (Chapter 6)

Table 5.2 — Merchant panel: endpoint-by-endpoint buildability.

5.3 Checkout flow and polling discipline

The checkout page walks the state machine of Figure 3. On the manual MFS path the page shows the merchant's number with a copy button and the exact amount, then collects the TrxID and sender phone after the customer pays from their own wallet app. Submission either completes instantly (the carrier SMS already matched, including the exact-amount check) or enters `awaiting_sms`, which is a calm waiting state, not an error. The page polls status on a capped backoff schedule and stops on any terminal state; the webhook to the merchant fires independently and remains the authoritative integration signal. Every inline rejection — a TrxID already claimed elsewhere, or an amount mismatch — renders as a retryable inline message, never a dead end.

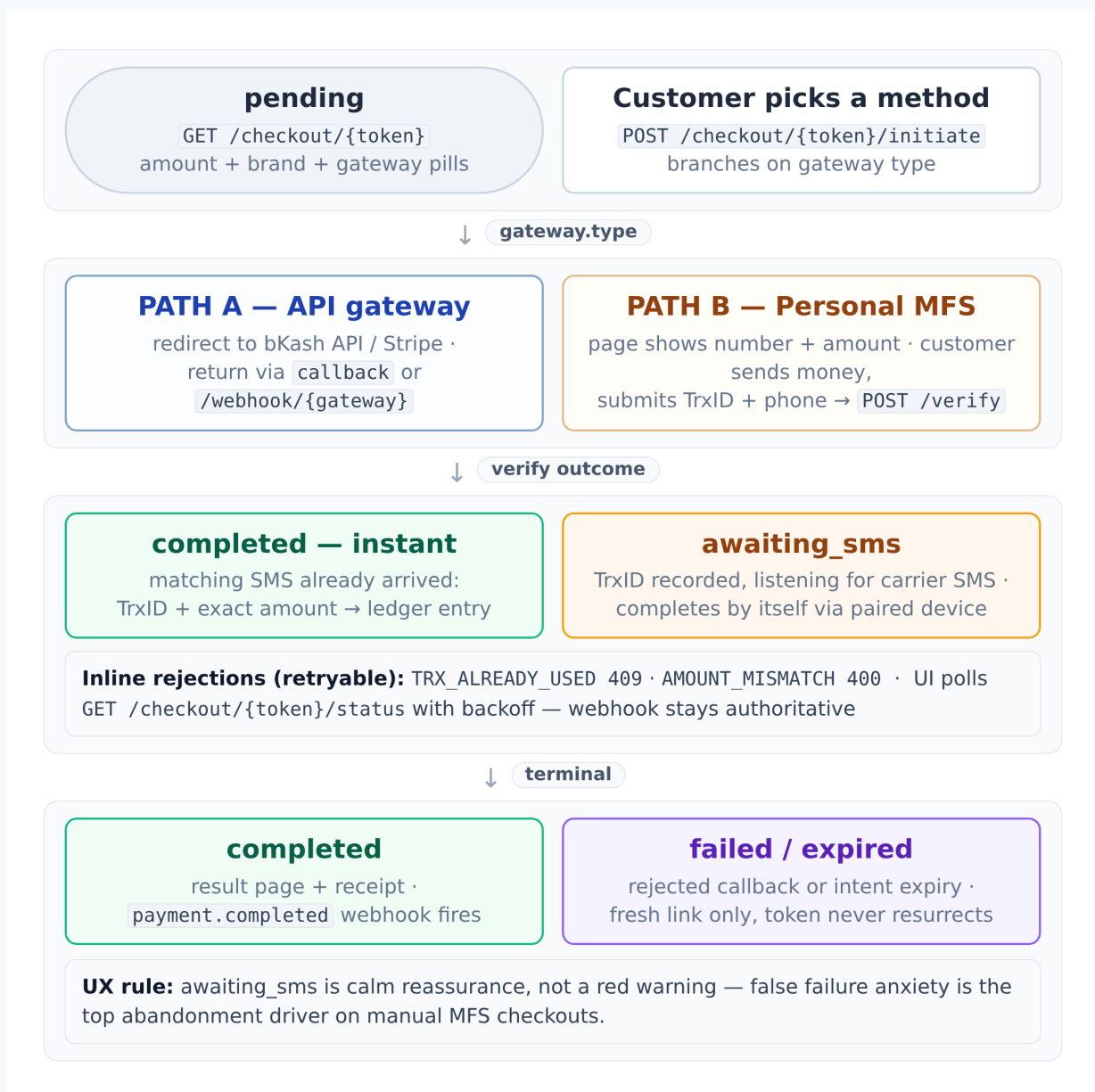


Figure 3 — Checkout state machine: API-gateway path and personal-MFS path, verified against `checkout.ts`.

Poll	Delay after previous	Cumulative	Behavior
1–3	2 s	6 s	fast loop for the instant-match case
4–6	5 s	21 s	steady while carrier SMS is in flight
7+	10 s (ceiling)	n/a	until terminal or tab close; stop on completed / failed / expired

Table 5.3 — Status polling schedule (convenience only; the webhook is authoritative).

6 • The Merchant-Write Gap and Remediation Paths

Three capability gaps survive verification, and all three sit exactly where the merchant panel's most valuable screens live. They are worth naming precisely, because every one of them changes what the UI can honestly promise. The evidence is structural, not anecdotal: writes to the manual-gateway credentials table exist only inside admin provisioning and the bootstrap path; pairing tokens are created only at provisioning (with a thirty-day expiry) and by the install wizard; and the analytics surface begins and ends at the trial balance.

6.1 The gaps, with evidence

ID	Gap	Evidence in source	Who feels it
G1	Merchant cannot write own gateway credentials (bKash personal number, API secrets)	op_manual_gateways written only in admin-api.ts provisioning and bootstrap.ts	the core 'configure my payment system' screen
G2	Merchant cannot generate a new device-pairing OTP	op_device_pairing_tokens written only at provisioning (30-day TTL) and install	phone pairing after expiry requires an operator
G3	No time-series analytics endpoints	only /ledger/trial-balance exists; no volume, success-rate, or cohort routes	dashboards aggregate client-side from /transactions

Table 6.1 — Verified gaps and where they bite.

6.2 Path A — the target-state merchant-scoped endpoints

The right long-term fix mirrors the security patterns the core has already proven. Gateway credential writes belong on the merchant surface behind write scope, stored with the same AES-256-GCM envelope used for claim staging, never echoed back on read (the catalog endpoint already models this: field labels, never values). Pairing OTP generation becomes a merchant-scoped endpoint that writes to the same token table with a shorter TTL and an audit event, rate-limited like the pairing route itself. Analytics gets a summary

endpoint parameterized by day-range, computed from transactions with the ledger as the tiebreaker. Each endpoint arrives with tests in the established `crypto-security.test.ts` idiom — round-trip, fail-closed, timing-safe — so the 260-test battery grows rather than dilutes.

Proposed endpoint	Method + scope	Stored how	Notes
<code>/api/v1/gateways/{id}/credentials</code>	PUT, write	AES-256-GCM envelope in D1/KV	write-only; read returns masked hints only
<code>/api/v1/devices/pairing-tokens</code>	POST, write	same token table, 15-min TTL	rate-limited; one active OTP per merchant
<code>/api/v1/analytics/summary?from&to;</code>	GET, read	computed	volume, success rate, per-gateway split; ledger-tied

Table 6.2 — Path A endpoint specs, aligned to existing crypto patterns.

6.3 Path B — admin-mediated v1 (ships today)

Path B keeps the honest contract the API actually exposes. The merchant panel renders the gateway-configuration screen as a request form: the merchant enters the number and secret, the panel stores a draft (client-side or in intent metadata) and shows it as pending; an operator completes the write in the admin console during provisioning-style editing, and the merchant sees the state flip to active on refresh. Pairing follows the same shape — the panel explains that an operator issues a fresh OTP, and surfaces the existing one's countdown while it is still valid. It is slower and involves a human, which is precisely why it is safe: no new attack surface ships on day one, and the eventual Path A migration replaces a form submission, not a security model. This blueprint's delivered merchant sample marks these screens with an explicit 'operator-assisted' badge so the limitation is visible in the interface itself rather than hidden in documentation.

Recommendation

Ship v1 on Path B; schedule Path A as the first backend milestone of Phase 2. Design the merchant screens now with the exact form fields Path A will submit, so the migration is a URL change plus a badge removal — not a redesign.

7 • After-Payment Flow: Callbacks, SMS Corroboration, Webhooks

Everything after the customer pays converges on one invariant: a transaction reaches completed exactly once, the ledger posts exactly once, and the merchant learns about it through a signed, retried, observable webhook. Three doors feed the same completion guard — the synchronous browser callback, the asynchronous gateway event with (merchant, gateway, event_id) dedup, and SMS corroboration on the manual path — and none of them can double-post because the LedgerDO serializes the journal entry. The frontend's role in all of this is deliberately modest: the checkout page polls for its own spinner, the merchant dashboard renders the delivery log, and nobody is invited to trust a page state over the webhook.



Figure 4 — The after-payment pipeline: ingestion, guard, ledger, fan-out, receiver duties.

7.1 The webhook contract the UI must teach

Property	Value (verified)	What the UI shows
Event	payment.completed	delivery log rows keyed by trx_id
Signature	HMAC-SHA256 over the raw request body	verification helper + copyable snippet
Headers	X-EdgePay-Signature, X-EdgePay-Timestamp	docs panel in the webhook settings screen
Retries	3 attempts, 60 s backoff (WEBHOOK_MAX_RETRIES / WEBHOOK_BACKOFF_MS)	per-delivery attempt timeline
Final failure	DLQ entry + page-level alert + webhook_dlq metric	red row with re-fire guidance
Self-test	POST /api/v1/webhooks/tests sends webhook.test	one-click test with live response log
Dedup	same event may legitimately arrive twice across doors	idempotent-upsert guidance in docs

Table 7.1 — Outbound webhook contract and where each property surfaces in the UI.

7.2 Awaiting-SMS is a design problem, not a status

On personal MFS rails the customer submits a TrxID before the carrier SMS has landed, and the transaction legitimately waits. The single most influential UX decision in the whole checkout is rendering that wait as calm reassurance — a soft amber state with a progress metaphor like 'listening for your carrier's confirmation' — rather than a red warning. False failure anxiety is the top abandonment driver on manual checkouts in this market, and every rejection the API returns is deliberately retryable: a claimed TrxID or an amount mismatch is inline guidance to re-check the wallet app, not a dead end. The delivered checkout sample implements this state with an animated listening indicator, an honest elapsed-time counter, and a copyable reference so the customer can close the tab and still reconcile later.

8 • Frontend Stack: Astro Islands, GSAP, and the Design System

The stack choice follows the deployment target. Cloudflare Workers serve static assets at the edge with zero cold-start, and Astro is the framework built for exactly that contract: pages ship as static HTML with no JavaScript by default, and only the components that genuinely need interactivity hydrate as islands. A checkout page therefore costs almost nothing on first paint, while the gateway selector, the TrxID form, and the status poller hydrate on demand with small, isolated scripts. The merchant and admin consoles follow the same discipline — server-rendered tables and cards, hydrated toolbars and drawers — which keeps every surface inside the performance budgets of Chapter 9 without fighting the framework.

8.1 Motion language with GSAP

GSAP earns its bytes by doing work CSS transitions cannot: timeline sequencing for state changes, staggered entrance, number count-ups, and the SVG stroke-draw success checkmark. The discipline is strict. The library loads deferred and cross-origin with SRI, animations are gated behind prefers-reduced-motion so motion is an enhancement rather than a requirement, and durations stay in the 120–300 ms band with expo-out easing — fast enough to feel responsive on mid-range Android, slow enough to be legible. Stagger steps of 40–60 ms keep lists feeling organized instead of circus-like, and nothing animates a layout property that would cause shift.

Pattern	Spec	Where it is used
Entrance stagger	y: 12→0, 220 ms expo-out, 45 ms stagger	table rows, gateway pills, KPI cards
State transition	timeline 260–300 ms: fade out → swap → settle	checkout steps, drawer open/close
Count-up	snap to tabular numerals, 500 ms, 1 decimal	amounts, KPI values
Success draw	SVG strokeDashoffset, 420 ms, single accent color	completed checkmark
Haptic accent	navigator.vibrate on state boundaries	see 8.2 — paired with every pattern above

Table 8.1 — Motion specification shared by all three surfaces.

8.2 Haptic response

Haptics make a payment flow feel physically trustworthy on the phones where it actually runs, and the web primitive is one line behind a feature check. The kit implements a single haptic helper with distinct patterns per semantic event — a light tick on every button press, a two-pulse pattern on success, a heavier triple pattern on error — each no-op on desktop and each suppressed when the user prefers reduced motion. Vibration is paired with a visual state change, never used alone, so nothing depends on a sense the device may not have.

Haptic helper (from the delivered kit)

```
// packages/ui/src/haptics.ts
const canVibrate = 'vibrate' in navigator
&& !matchMedia('(prefers-reduced-motion: reduce)').matches;

export const haptic = {
  tap() { canVibrate && navigator.vibrate(8); },
  select() { canVibrate && navigator.vibrate([0, 10, 30, 10]); },
  success(){ canVibrate && navigator.vibrate([10, 40, 20]); },
  error() { canVibrate && navigator.vibrate([60, 40, 60]); },
};
```

8.3 Design tokens

One token set spans the three surfaces so a merchant moving from their dashboard to their customer's checkout page experiences one product. The neutrals are slate-family for text hierarchy, the brand blue carries all primary actions, and the single amber tone is reserved for the awaiting/waiting semantics of the MFS flow. Gateway brand colors are functional data, not decoration: bKash pink, Nagad orange, and Rocket purple identify the payment rails customers already know, and appear only on gateway pills and their selected states. Amounts render with tabular numerals and the BDT formatting the market expects.

Token	Value	Role
--ink / --paper	#0F172A / #F8FAFC	primary text on primary background
--brand / --brand-ink	#1D4ED8 / #1E3A8A	actions, links, emphasis
--surface / --line	#FFFFFF / #E2E8F0	cards and hairlines
--awaiting	#B45309 on #FFF7ED	awaiting_sms and pending semantics (amber)
--success / --error	#065F46 / #B31212 (muted)	terminal states only
--bkash / --nagad / --rocket	#E2136E / #F6921E / #8C3494	gateway identity (functional color)

Table 8.2 — EdgePay frontend tokens (mirrored in the delivered samples).

8.4 Typography and locale posture

UI text uses the system font stack first (zero download, native rendering on every Android in the market) with Inter as the progressive enhancement where available; numerals are always tabular so aligned amounts stay aligned. Currency renders in the en-BD convention with the taka sign, and every string in the samples is a lookup away from a bn translation table because a meaningful share of checkout traffic will be Bengali-first. Dates and times format for Asia/Dhaka explicitly rather than trusting device locale, so a merchant reading a transaction log never has to guess which timezone a timestamp means.

9 • Performance and Security Budgets

Budgets are set per surface because the surfaces differ in what they owe the user: checkout owes speed to strangers on mid-range phones over patchy networks, while the consoles owe density and scan-ability to authenticated operators on desktop. The checkout budget is the strict one — sub-second first paint of the shell, near-zero total blocking time, no layout shift when status changes — and the techniques are boring on purpose: inline critical CSS, system fonts, deferred GSAP, inline SVG icons instead of an icon font, aspect-ratio boxes everywhere, and content-visibility on below-fold sections. The delivered samples are built to these budgets so the numbers are demonstrated rather than aspirational.

Metric	Checkout	Consoles	Primary technique
LCP	≤ 1.8 s on Fast 3G	≤ 2.5 s	static shell, zero-JS first paint
TBT	≤ 150 ms	≤ 300 ms	islands hydrate on interaction
CLS	≤ 0.1 (status changes included)	≤ 0.1	fixed boxes, reserved space
JS shipped	≤ 45 KB gz (excl. CDN GSAP ~29 KB)	≤ 90 KB gz	islands, no framework runtime
Requests before paint	≤ 2	≤ 4	inline critical CSS + system fonts

Table 9.1 — Per-surface performance budgets.

9.1 Security posture per worker

The checkout surface inherits and extends the core's EDGE-P0-006 header set: a CSP with default-src 'self', no framing, no third-party script origins except the specific gateway hosts a redirect requires, and nosniff plus strict referrer policy. All user- and gateway-supplied strings — callback query parameters, gateway names, error messages — render through textContent or escaped interpolation, never innerHTML, because the callback path is precisely where an attacker with control of a redirect shape would plant markup. The merchant worker stores nothing sensitive in the browser: no keys in localStorage, no PII beyond the session cookie, with CSRF tokens on every mutating form. The admin worker keeps its service-token pair in worker secrets only, and its UI never renders it. Rate limits get honest treatment everywhere: a 429 renders as a countdown with a retry affordance, never a silent spin.

Control	Checkout	Merchant	Admin
CSP frame-ancestors	'none'	'none'	'none'
Script origins	'self' + gateway redirect hosts	'self'	'self'
Cookies	none	httpOnly · Secure · SameSite=Strict + CSRF	httpOnly session
Secrets in worker	none	session encryption key	Access token pair + admin key
XSS posture	textContent-only rendering of all externals	escaped interpolation	escaped interpolation

Table 9.2 — Security controls per surface.

10 • Implementation Roadmap

The phasing orders risk backwards: backend contract fixes that are cheap and unblock everything come first; the public checkout — the highest-value, highest-traffic surface — ships next with zero dependency on the merchant-write gap; the merchant panel follows on Path B so it launches against endpoints that exist; the admin console completes the platform story; and the Path A endpoint migration closes the loop by removing the operator-assisted badges. Each phase carries explicit entry and exit criteria so 'done' is checkable, and each extends the existing test battery rather than starting a parallel one — the repository's vitest-pool-workers setup in workerd is the right harness for the BFF and proxy logic too.

Phase	Scope	Entry criteria	Exit criteria
0 • Contract fixes	regenerate OpenAPI; JSON intent negotiation; analytics summary endpoint	core green at 260 tests	spec matches router; contract tests pass
1 • Checkout	Astro checkout kit on edgepay-checkout; custom domains; state machine + haptics	Phase 0 JSON intent live	end-to-end bKash personal + API-gateway payment on staging
2 • Merchant	BFF session, transactions, payment links, webhooks, keys; Path B request forms	Phase 1 stable	merchant completes a full day's workflow unassisted
3 • Admin	merchants + claim flow, devices, SMS, domains, refunds, reconcile, ledger view	Phase 2 stable	operator provisions and supports a merchant end-to-end
4 • Path A	merchant-scoped credential writes, pairing OTP, analytics GA	Phase 3 adopted	badges removed; crypto-security tests extended and green

Table 10.1 — Phase plan with checkable entry and exit gates.

Two risks deserve standing attention. First, the stale-spec drift that already bit this review will recur unless the OpenAPI document is regenerated in CI whenever route registrations change; a small check that diffs the served document against the router keeps the contract honest forever. Second, the admin-mediated Path B workflows are only as good as their visibility — if an operator forgets a pending gateway request, the merchant panel should nag, which costs one query on the admin console's landing screen. Neither risk is architectural; both are habits, which is exactly the right note to end on: the platform has spent nine audit rounds buying correctness, and this frontend plan is designed to inherit that posture instead of spending it.